AGROSIERRA

Examen Parcial II · Unidad 2

AgroAPI + AgroCampo
Entrega técnica y evidencias verificadas

Dato	Detalle
Estudiante	Axel Perez
Asignatura	Desarrollo de Aplicaciones Móviles
Caso	AgroSierra: labores de campo de una cooperativa agrícola
Fecha de preparación	27 de octubre de 2026 · America/Lima
Código / grupo	No proporcionados
Plataforma evaluada	Android con módulo común KMP
Solicitud preparada	SC-A · filtro Solo campañas abiertas

Verificación automatizada: 9 pruebas del backend y 7 pruebas compartidas; 0 fallos registrados. Además, 13 escenarios HTTP ejecutados contra AgroAPI real.

Se comprobó una creación en modo avión, persistencia de la fila pendiente y reconexión sin duplicación. Las capturas corresponden al emulador y las respuestas HTTP se guardan sin tokens.

Transparencia: el código fue preparado con asistencia de IA. Los commits nuevos tienen su fecha real; no acreditan desarrollo durante semanas previas. La defensa individual y la asignación oficial del cambio corresponden al estudiante.

1. Solución y arquitectura

AgroAPI usa Spring Boot, Kotlin, Spring Security, JPA y una base H2 persistente en disco. AgroCampo adapta la estructura de PharmaMobil y usa Compose, Koin, Ktor y SQLDelight. No hay listas de negocio incrustadas en la aplicación.

Capa	Responsabilidad y archivos
Backend: seguridad	SecurityConfig, JwtAuthenticationFilter y JwtService: firma, expiración y authorities; AuthController: login y usuario actual.
Backend: negocio	AgroServicio aplica RN-01 a RN-04 y transacciones. Los repositorios JPA acceden a Lote y Labor.
App: dominio	Lote, Labor, Sesion, TipoLabor, ErrorApi, AgroRepository y casos de uso sin anotaciones JSON.
App: datos	AgroApi + DTO + mappers; BaseLocal y AgroDb.sq; AgroRepositorioOffline coordina la cola.
App: presentación	AgroViewModel expone StateFlow y UiState. AgroApp dibuja login, padrón, detalle, formulario y perfil.
Android	AndroidSqliteDriver, AES-GCM/Keystore, monitor de conectividad y módulos Koin.

Recorrido: tabla del servidor -> repositorio JPA -> servicio -> controlador -> JSON -> DTO Ktor -> mapper de dominio -> SQLDelight -> Flow -> ViewModel -> Compose.

La pantalla obtiene datos de SQLite. Una sincronización descarga y escribe en la base; no sustituye la fuente de datos de la pantalla por una respuesta de red.

2. Contrato REST y autorización

Método / ruta	Privilegio	Éxito
POST /api/v1/auth/login	Público	200
GET /api/v1/usuarios/me	Autenticado	200
GET /api/v1/lotes	LOTE_LEER	200
GET /api/v1/lotes/{id}	LOTE_LEER	200
GET /api/v1/lotes/{id}/labores	LABOR_LEER	200
POST /api/v1/labores	LABOR_CREAR	201
PUT /api/v1/labores/{id}	LABOR_EDITAR	200
DELETE /api/v1/labores/{id}	LABOR_ELIMINAR	204
POST /api/v1/lotes/{id}/cerrar-campania	CAMPANIA_CERRAR	200
POST /api/v1/usuarios	USUARIO_GESTIONAR	201

Paginación: pagina >= 0, tamanio entre 1 y 100; envoltorio contenido, pagina, tamanio, totalElementos, totalPaginas, ultima. Lotes filtra por valle y campaniaCerrada. Labores ordena por fecha e ID descendentes.

Labor: loteId, tipo, fecha ISO, cantidad y observacion. El responsable procede del usuario autenticado. Idempotency-Key es una extensión opcional del POST para evitar duplicados en reintentos.

Todo error utiliza timestamp, status, error, message, path y validationErrors. DELETE 204 no se deserializa.

3. Privilegios y reglas de negocio

Privilegio	Técnico	Jefe de campo	Administrador
LOTE_LEER	Sí	Sí	Sí
LABOR_LEER	Sí	Sí	Sí
LABOR_CREAR	Sí	Sí	No
LABOR_EDITAR	Propias < 24 h	Todas	No
LABOR_ELIMINAR	No	Sí	No
CAMPANIA_CERRAR	No	Sí	No
USUARIO_GESTIONAR	No	No	Sí

Regla	Aplicación en el servicio
RN-01	Campaña cerrada: registrar o corregir labor devuelve 409.
RN-02	Sin política de edición global: debe ser autor y no haber pasado 24 horas desde registradoEn. Rechazo 403.
RN-03	Cerrar sin al menos una COSECHA devuelve 409.
RN-04	Fumigación superior a hectáreas × dosis máxima devuelve 400 con validationErrors.cantidad.

Los controladores verifican privilegios con @PreAuthorize. La política edicionGlobal vive en Rol y evita comparar nombres de rol en el controlador. AgroServicio vuelve a aplicar propiedad y plazo de edición dentro de la operación.

Semillas: 8 lotes en Mantaro, Sagrado y Colca; lotes 7 y 8 cerrados; 20 labores, al menos 2 por lote abierto; lote 2 sin cosecha para demostrar RN-03. Usuarios tecnico1, jefe1 y admin, contraseña de laboratorio Agro2026*, cifrada con BCrypt en la base.

4. Seguridad y respuestas reales

El JWT HS512 dura una hora y transporta ID de usuario, privilegios, iat y exp. La firma se verifica antes de atender rutas protegidas. La clave se genera fuera del código y no se distribuye. El almacén Android cifra la sesión con AES-GCM y conserva la clave en Keystore.

Escenario ejecutado	HTTP esperado / observado
Sin token	401 / 401
Firma alterada	401 / 401
Token firmado vencido	401 / 401
Paginacion	200 / 200
Admin no registra	403 / 403
Tecnico no cierra	403 / 403
RN01 lote cerrado	409 / 409
RN03 lote sin cosecha	409 / 409
RN04 cantidad excesiva	400 / 400
Crear labor	201 / 201
Editar labor	200 / 200
Eliminar sin cuerpo	204 / 204
Recurso inexistente	404 / 404

Fuente verificable: evidencias/api-resultados.json. Los datos son respuestas del proceso AgroAPI, no una simulación. Los escenarios automatizados no requieren conservar el bearer.

5. Rechazos 403 y 400 documentados

POST /labores -> 403

Admin no registra


```
{
  "timestamp": "2026-10-27T14:41:59.151957",
  "status": 403,
  "error": "Forbidden",
  "message": "No tiene el privilegio requerido",
  "path": "/api/v1/labores",
  "validationErrors": {}
}
```


POST /labores -> 400

RN04 cantidad excesiva


```
{
  "timestamp": "2026-10-27T14:41:59.296714",
  "status": 400,
  "error": "Bad Request",
  "message": "Datos inválidos",
  "path": "/api/v1/labores",
  "validationErrors": {
    "cantidad": "La dosis máxima del lote es 4.0"
  }
}
```

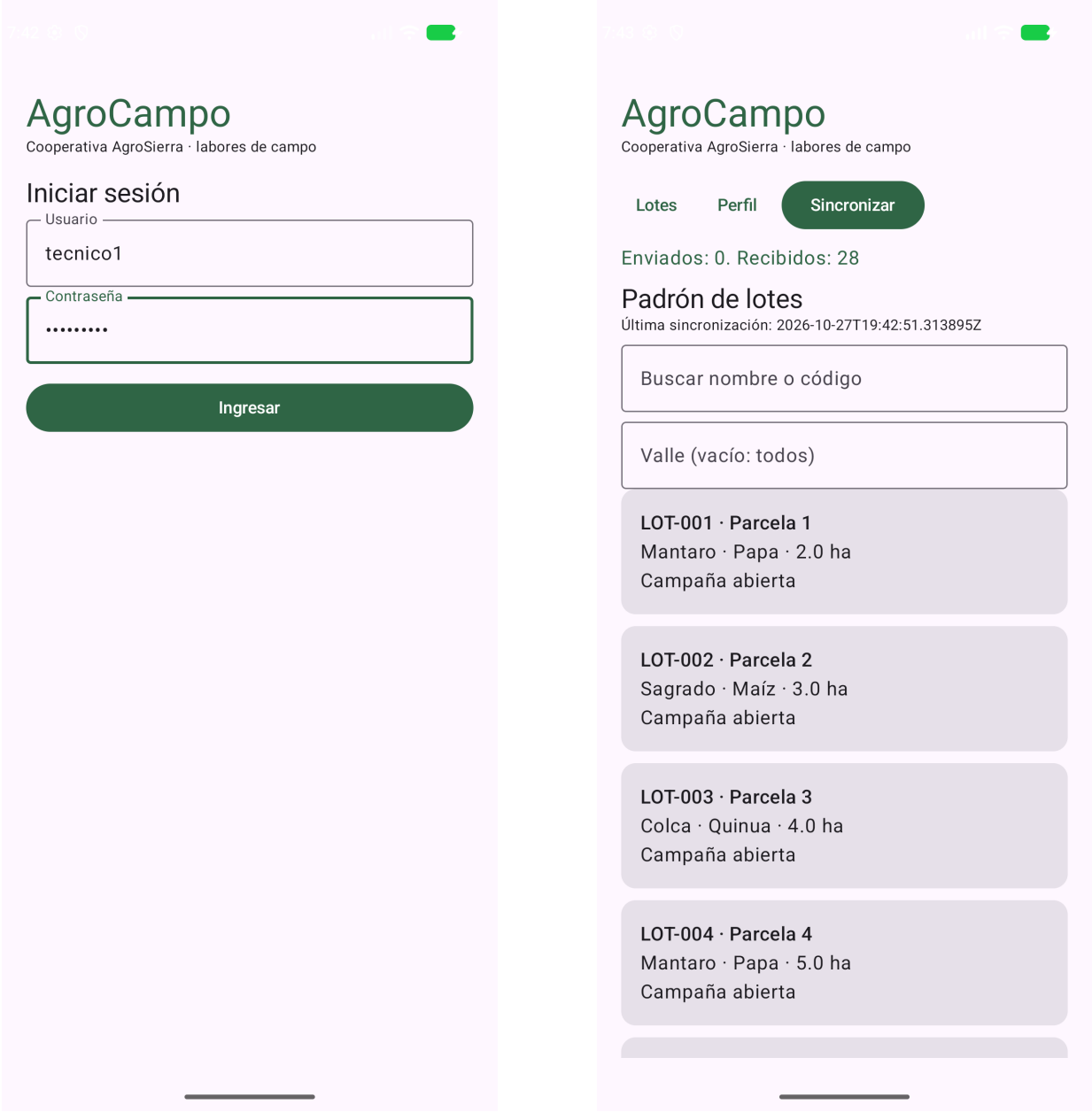

GET /lotes -> 401

Token firmado vencido


```
{
  "timestamp": "2026-10-27T14:41:59.043770600",
  "status": 401,
  "error": "Unauthorized",
  "message": "Sesión ausente, inválida o vencida",
  "path": "/api/v1/lotes",
  "validationErrors": {}
}
```


Son transcripciones de respuestas HTTP reales guardadas en JSON. No son capturas de Postman. La colección adjunta permite volver a provocar estos códigos desde esa herramienta.

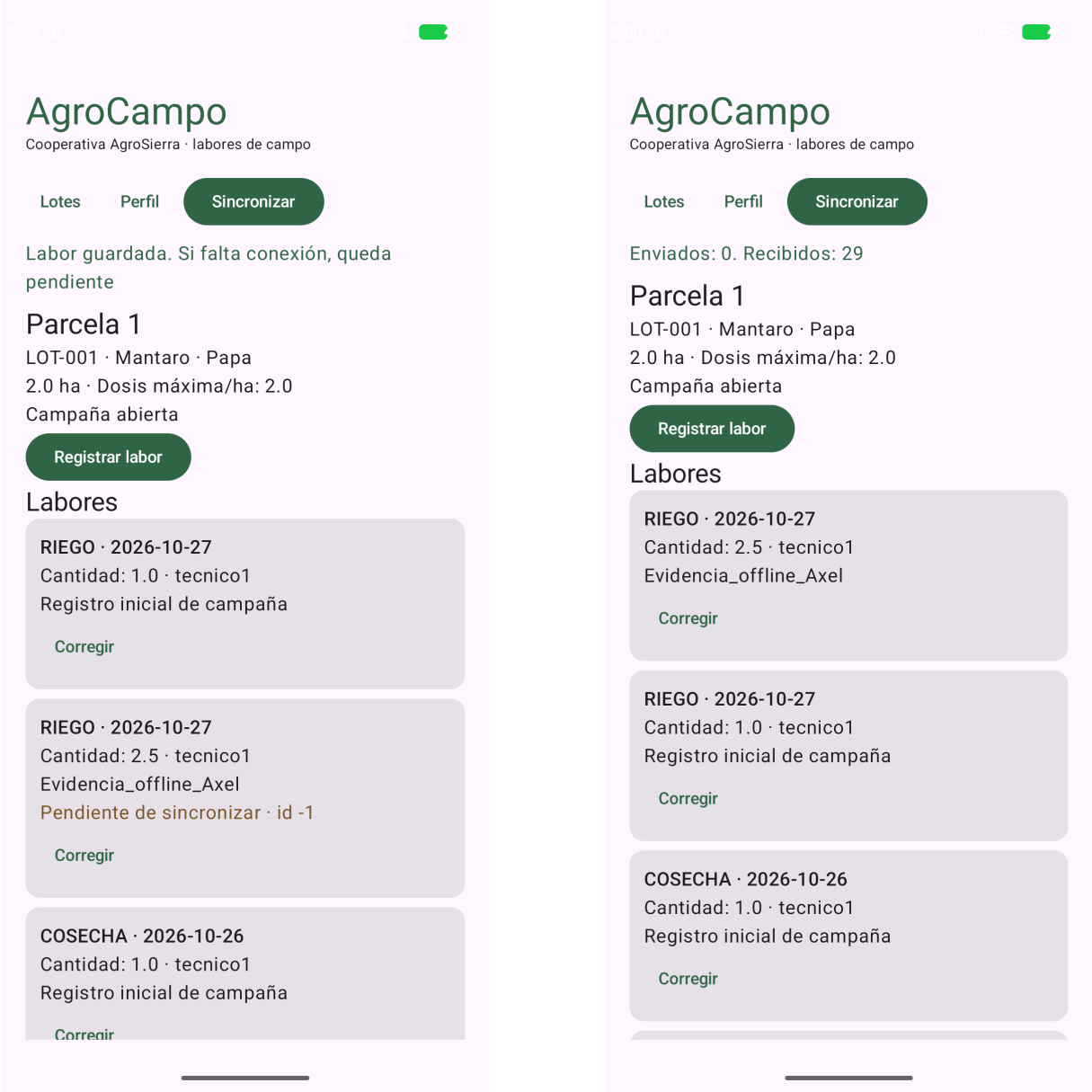
6. Aplicación Android en ejecución



Inicio de sesión con contraseña oculta | Padrón cargado desde la base local

Una única instancia de HttpClient usa URL centralizada, JSON tolerante y timeout. Logging oculta Authorization. El ViewModel expone estados de pantalla y operación; una mutación no vacía el listado existente.

7. Sin conexión y reconexión



Modo avión: fila pendiente con ID -1 | Después de reconectar: fila sincronizada

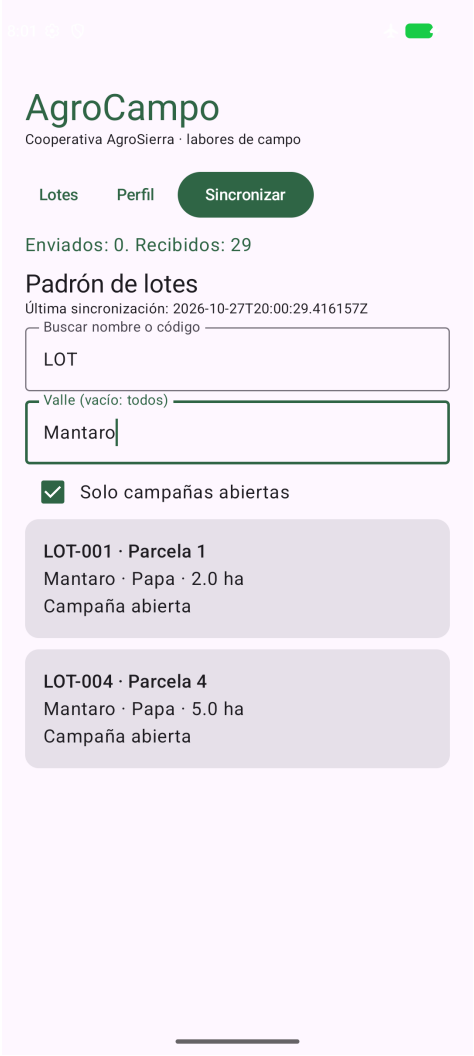
Consulta posterior al servidor: observación Evidencia_offline_Axel, ID definitivo 22, cantidad de coincidencias 1. La respuesta completa está en evidencias/reconexion.json.

La subida precede a la descarga. La sustitución provisional/definitiva es atómica en SQLite y la clave idempotente permite repetir un POST sin crear otra fila. Un Mutex evita sincronizaciones superpuestas.

8. Solicitud de cambio SC-A

Solicitud: agregar Solo campañas abiertas al padrón, combinado con valle y búsqueda y funcional sin conexión. Se prepara SC-A por no haberse comunicado otra letra asignada.

La consulta SQL ya admite soloAbiertas. El cambio conecta ese parámetro a UiState y al ViewModel, agrega Checkbox en Compose y verifica combinaciones en una base en memoria. El composable no filtra ni recorre la lista para aplicar reglas.



Filtro de campañas abiertas funcionando en modo avión

Rama: sc-a-perez. Commits: propagar filtro -> control visual -> prueba y documentación. Son commits de la fecha real de preparación; no se presentan como trabajo durante una evaluación en vivo que todavía no se ha acreditado.

9. Pruebas y trazabilidad

Suite	Ejecutadas	Fallos
AgroAPI / test	9	0
AgroCampo / commonTest en JVM	7	0
Escenarios HTTP del proceso real	13	0

Comandos: AgroAPI/gradlew.bat -p AgroAPI test ; AgroCampo/gradlew.bat -p AgroCampo :shared:jvmTest. Los reportes HTML se incluyen en evidencias/reportes y los XML originales permiten revisar cada resultado.

MockEngine sustituye la red; JdbcSqliteDriver en memoria sustituye el almacenamiento en pruebas. La aplicación de producción usa OkHttp, AndroidSqliteDriver y backend real. No se inyectan repositorios de prueba en Android.

Los repositorios locales incluyen main, develop, ramas feature, fusiones --no-ff y etiqueta v1.0-unidad2. AgroCampo conserva la historia de la base del docente identificada como tal. Solo los commits nuevos corresponden a esta adaptación.

No se puede acreditar retrospectivamente el trabajo distribuido a lo largo de la unidad ni sustituirlo con ramas creadas hoy. Las solicitudes de incorporación remotas solo pueden existir después de publicar los repositorios; no se afirma que ya estén cerradas.

Historial Git: AgroAPI

```
*   d19a40b merge: preparar producto de la unidad 2
|\
| *   f07c002 merge: integrar documentación de entrega
| |\
| | * 2dff9e3 docs: explicar arquitectura, defensa y tareas de verificación
| |/
| *   ce2a71b merge: integrar pruebas automatizadas
| |\
| | * dbb492e test: verificar seguridad y sincronización con escenarios reproducibles
| |/
| * 7a0cab6 merge: integrar implementación AgroSierra
|/|
| * 3f78591 feat(api): proteger el CRUD con JWT, privilegios y reglas del caso
|/
* a75798d docs: iniciar entrega AgroAPI y explicar alcance asistido
```

Salida real de git log --graph --oneline --all. El historial base de PharmaMobil no acredita autoría del estudiante sobre el código del docente.

Historial Git: AgroCampo

```
* 3a125d7 test(sc-a): verificar filtros combinados sin conexión y documentar la demostración
* 72f88d5 feat(sc-a): ofrecer el filtro combinado con valle y búsqueda
* 2d93ad5 feat(sc-a): propagar el filtro de campañas abiertas a la consulta local
| *   3f18f7b merge: preparar producto de la unidad 2
| |\
| |/
|/|
* |   cc3708e merge: integrar documentación de entrega
|\ \
| * | 8b68de8 docs: explicar arquitectura, defensa y tareas de verificación
|/ /
* |   a4daf2e merge: integrar pruebas automatizadas
|\ \
| * | c98f29d test: verificar seguridad y sincronización con escenarios reproducibles
|/ /
* |   44a8b5c merge: integrar implementación AgroSierra
|\ \
| |/
|/|
| * 0a77267 feat(agro): integrar seguridad, reglas y persistencia offline
|/
* fe7f420 opciones
* a51a325 clean
* c15540b Agrega tutorial de los cambios de la Sesion 5
* a64459e Rediseña la interfaz con identidad propia y estados con contenido
* 3a4bf01 Reorganiza el modulo de Productos en capas Clean + MVVM con Koin
* 947ed3c clase04
* 0d522d2 tabs
* 64a4d94 cliente
* 35e589b cliente
```

Salida real de git log --graph --oneline --all. El historial base de PharmaMobil no acredita autoría del estudiante sobre el código del docente.

10. Lista de cotejo y límites de evidencia

N.º	Criterio	Resultado
1	Backend inicia con un comando	Verificado
2	Login firmado y rechazo de contraseña	Verificado
3	Rutas protegidas sin token	Verificado
4	Token alterado o vencido -> 401	Verificado
5	Sin privilegio -> 403 y sesión conservada	API verificada; conducta implementada
6	Paginación y tamaño	Verificado
7	400 con detalle por campo	Verificado
8	Token seguro en Android	Implementado; cifrado AES-GCM
9	Authorization centralizada	Verificado en código y llamadas
10	Logout borra token	Implementado
11	UI respeta privilegios	Implementado y demostrado con técnico
12	Listado desde Flow local	Verificado
13	Datos en modo avión	Verificado en emulador
14	Registro pendiente e ID provisional	Verificado: -1
15	Reconexión sin perder ni duplicar	Verificado: ID 22, una coincidencia
16	commonTest pasa	Verificado
17	Ramas y commits en varias fechas de la unidad	No acreditable con esta entrega
18	Funcionalidades por fusión a main	Ramas locales actuales
19	Convenciones y etiqueta	Preparadas en Git local
20	Tres commits de cambio durante examen	Tres commits preparados; bloque en vivo no acreditado

11. Guía de defensa individual

Estas respuestas describen el código entregado. Ensaya mostrando cada archivo y ejecutando la operación; no afirmes haber hecho pruebas que no puedas repetir.

1. ¿Dónde se verifica el privilegio? En AgroAPI, config/SecurityConfig.kt exige autenticación y web/AgroController.kt usa @PreAuthorize(hasAuthority(...)). config/JwtAuthenticationFilter.kt construye las authorities desde el usuario de la base. Prueba POST /labores con admin: 403 aunque se invoque fuera de Android. Ocultar botones solo mejora la experiencia.

2. ¿Qué lleva el JWT? config/JwtService.kt firma HS512 con clave persistente, coloca sub (ID), privilegios, iat y exp. La carga se codifica en Base64URL y no está cifrada. El filtro verifica firma y expiración; un token vencido devuelve 401. No existe refresh.

3. ¿Dónde se guarda? androidApp/.../AlmacenSeguroAndroid.kt cifra la sesión con AES-GCM. SharedPreferences contiene IV y texto cifrado; la clave AES no sale de Android Keystore. Las preferencias sin cifrar revelarían un bearer que permite actuar como su titular.

4. ¿Quién añade Authorization? shared/.../data/remote/ClienteHttp.kt, dentro de defaultRequest, lee el almacén y añade Bearer solo a rutas protegidas. Centralizarlo evita olvidos y duplicación. Logging sanea la cabecera y se desactiva en release.

5. ¿401 o 403? MapeoErrores.kt traduce cada código. El callback 401 de AgroRepositorioOffline.red() borra el almacén y el StateFlow de sesión. El 403 se convierte en SinPrivilegio, muestra mensaje y mantiene la sesión. Las credenciales incorrectas tienen su propio mensaje durante el login.

6. Recorrido del dato: backend dominio/Entidades.kt -> repositorio/Repositorios.kt -> servicio/AgroServicio.kt -> web/AgroController.kt -> JSON -> aplicación AgroApi.kt -> dto/Dto.kt -> dominio() -> AgroRepositorioOffline.kt -> BaseLocal.kt/AgroDb.sq -> Flow -> AgroViewModel.kt -> AgroApp.kt -> Compose.

7. ¿Por qué observar SQLite? La red puede faltar. BaseLocal.lotes() usa asFlow().mapToList() sobre SQLDelight. La UI observa siempre la base; sincronizar solo la modifica. Muestra el listado con modo avión.

8. Registro offline: guardar() asigna un ID negativo, guarda pendiente y operación CREAR, e intenta enviar. Sin conexión la fila queda visible. Su claveCliente se conserva para reintentar el mismo POST. Muestra la fila y la consulta siguienteIdLocal.

9. Orden de sincronización: Mutex -> enviar pendientes -> descargar todas las páginas -> reemplazar sincronizadas en una transacción -> guardar fecha. Si se descargara y borrara primero, se podría perder lo creado sin red. La sustitución provisional/definitiva es atómica y el POST idempotente evita duplicados ante respuestas perdidas.

10. ¿Cómo se prueba? commonTest/.../RepositorioTest.kt usa MockEngine para respuestas y fallos; jvmTest/.../BasePrueba.kt aporta JdbcSqliteDriver en memoria. subirAntesDeBajar registra las rutas y verifica su orden. registrarSinRed comprueba la fila pendiente. Ejecuta :shared:jvmTest y muestra el reporte.

11. Validación por campo: AgroServicio.validar genera Fallo(400, ..., campos); ManejadorDeErrores serializa validationErrors; ErrorDto lo recibe; MapeoErrores produce ErrorApi.Validacion; ViewModel copia campos a UiState y Campo los dibuja bajo cada entrada. Provoca una fumigación de cantidad superior al máximo.

12. Rol nuevo: agrega el rol, sus privilegios y su política edicionGlobal a las tablas por una migración/semilla idempotente. Usuarios se asignan al rol. No cambies controladores ni compares strings de nombres de rol. El login y /usuarios/me ya devuelven privilegios y la UI los usa. Para SC-B: SUPERVISOR_CALIDAD = LOTE_LEER + LABOR_LEER + LABOR_EDITAR, edicionGlobal=true, sin CAMPANIA_CERRAR.

13. Git y autoría: muestra git log --graph --oneline --all y la rama sc-a-perez. Explica qué se adaptó desde PharmaMobil y qué se agregó con asistencia. No inventes un conflicto: si no ocurrió, dilo y explica cómo resolverías uno en la rama feature antes de fusionar. Los commits actuales no reemplazan el historial de trabajo previo de la unidad.

12. Entrega y referencias

Archivos: README general; AgroAPI y AgroCampo con sus README; binarios JAR/APK; colección Postman; evidencias PNG/JSON; reportes de pruebas; gráficos Git; guía de defensa y demostración; rama sc-a-perez.

Para una instalación nueva usa el código fuente y una base vacía del laboratorio: no se incluyen datos del servidor ni claves de firma. El usuario debe aportar su código y grupo académico si el formato de entrega los exige.

GitHub: Repositorios de entrega en la cuenta Axeeeel8883:

- Android: https://github.com/Axeeeel8883/examen-axel-perez-2
- Backend: https://github.com/Axeeeel8883/examen-axel-perez-2-api

El producto base se encuentra en main; el cambio preparado se publica en sc-a-perez de Android. Las fusiones remotas se realizan por solicitudes de incorporación. Los commits corresponden a la fecha real de preparación, no a semanas anteriores.

Material del curso consultado: sesiones 7 (Ktor/REST), 8 (CRUD/errores), 9 (capacidades por plataforma), 10 (JWT/almacén seguro), 11 (offline-first), y examen parcial II práctico versión A, caso AgroSierra.

Proyecto base: https://github.com/dreyna/pharmaMobil

Documentación técnica: https://ktor.io/docs/ ; https://sqldelight.github.io/sqldelight/2.1.0/ ; https://docs.spring.io/spring-security/reference/ ; https://kotlinlang.org/docs/multiplatform.html

La implementación final usa versiones compatibles comprobadas mediante compilación. El trabajo incluye asistencia de IA, que debe explicarse durante la defensa. No se inventaron capturas de herramientas externas, resultados de tests, historial previo ni una asignación oficial de solicitud de cambio.